

SudokuSolver

User manual

Bilingual Sudoku solver, generator and tutor

Version 1.3.0

MIT | github.com/ARP273-ROSE/SudokuSolver

Contents

1 Overview **2**

2 Installation **2**

2.1 The installer (recommended) . . . 2
2.2 Updates . . . 2
2.3 Problem reports . . . 3
2.4 Running from the sources . . . 3

3 The interface **3**

3.1 Main window . . . 3
3.2 Keyboard . . . 4
3.3 Cell states . . . 4

4 Features **4**

4.1 Generating a puzzle . . . 4
4.2 Hint . . . 4
4.3 Next step . . . 5
4.4 Solve . . . 5
4.5 Check . . . 5
4.6 Candidates . . . 5
4.7 Reset . . . 5
4.8 File menu . . . 5
4.9 Language menu . . . 5
4.10 Help menu . . . 5

5 Under the hood **6**

5.1 Solver architecture . . . 6
5.2 Human-style techniques . . . 6
5.3 Generator . . . 6
5.4 Threading and anti-freeze . . . 6
5.5 Crash reports . . . 7

6 Troubleshooting **7**

6.1 “Python not found” on Windows . . . 7
6.2 “Installation failed” after a Python upgrade . . . 7
6.3 The generator is slow on Expert . . . 7
6.4 I imported a puzzle and Solve says “no solution” . . . 7

6.5 Where are my settings stored? 7
6.6 The app crashed — what should I do? 8

7 FAQ **8**

8 Credits **8**

1 Overview

SudokuSolver is a desktop application that solves, generates and teaches Sudoku puzzles. It is written in Python with a PyQt6 graphical interface and runs on Windows, Linux and macOS. The user interface is fully bilingual (French / English) and every widget has a tooltip in both languages.

Key features

- Fast solver with bitmask constraint propagation and MRV backtracking
- 8 human-style techniques with step-by-step explanations
- Puzzle generator with 4 calibrated difficulty levels
- Interactive 9×9 grid, pencil marks, real-time conflict highlighting
- Cross-platform (Windows, Linux, macOS)
- Crash reports anonymised automatically (no usernames, no paths)

2 Installation

2.1 The installer (recommended)

Ready-to-use packages, Python included, are published on the releases page. No administrator rights are needed: everything installs into your own profile.

System	File and first launch
Windows	`SudokuSolver-Setup.exe`. If SmartScreen warns that the installer is unsigned: *More info*, then *Run anyway* (only once).
macOS	`SudokuSolver-X.Y.Z-macos-arm64.dmg` (Apple Silicon) or `-macos-x86_64.dmg` (Intel): drag the application into Applications, then the first time *right-click* > *Open*.
Linux	`SudokuSolver-X.Y.Z-linux-x86_64.tar.gz` (or `-linux-arm64`): unpack, then run `./installer.sh` (installs into `~/.local/share`, adds the menu entry, no sudo).

This address always points at the latest Windows installer:

`https://github.com/ARP273-ROSE/SudokuSolver/releases/latest/download/SudokuSolver-Setup.exe`

2.2 Updates

Once installed, SudokuSolver looks for a new version at start-up and on demand (**Help > Check for updates**). It downloads only the small application archive from GitHub (any other address is refused), applies it and restarts — on Windows, macOS and Linux alike. The release notes are shown in the language of the interface.

2.3 Problem reports

At first launch, SudokuSolver asks whether it may send a short technical report automatically when it crashes or freezes (version, system, error trace; nothing else). The choice can be changed at any time in the **Help** menu, where **Report a problem…** also lets you write a report yourself.

2.4 Running from the sources

Requirements: Python 3.10 or newer, `PyQt6` $\geq$ 6.5. On Windows, `launch.bat` detects Python, creates a virtual environment in `%APPDATA%\SudokuSolver\venv` and starts the application; `launch.bat -repair` rebuilds it. On Linux and macOS, `./launch.sh` does the same in `$XDG_DATA_HOME/SudokuSolver/venv`. By hand:

```
pip install -r requirements.txt
python main.py
```

3 The interface

3.1 Main window

The window has four main areas:

1. **Top bar** — difficulty selector and “New puzzle” button.
2. **Grid** — the interactive 9×9 board.
3. **Action bar** — Hint, Next step, Solve, Check, Candidates, Reset.
4. **Explanation label** and **status bar** below.



The main window, with pencil marks



A hint: the cell in green, the explanation underneath

3.2 Keyboard

Key	Action
Arrow keys	Move selection
1–9	Enter digit in the current cell
0, Delete, Backspace	Clear cell
Mouse click	Select a cell

3.3 Cell states

- **Given** — original clue, cannot be edited, displayed in bold.
- **User entry** — digit you typed, displayed in a distinct colour.
- **Conflict** — cell contents violate a row, column or box rule; background turns red.
- **Candidates** — small digits showing possible values, toggled by the "Candidates" button.

4 Features

4.1 Generating a puzzle

Pick a difficulty in the combo box and click **New puzzle**. The generator creates a grid with rotational symmetry, guaranteed unique solution, and calibrates the clue count and hardest technique required to match the target difficulty.

Level	Clues	Hardest technique
Easy	36–45	Hidden Single
Medium	30–35	Hidden Pair
Hard	26–32	Box / Line Reduction
Expert	22–28	X-Wing

Generation runs in a background thread so the interface never freezes, even if the generator needs to retry several times to hit the target difficulty.

4.2 Hint

Click **Hint** to see the next logical move. The engine picks the simplest applicable technique, highlights the pattern cells in green and the eliminated candidates in red, and explains *why* the move works.

Tip

If no technique applies, the engine falls back to a guess based on the backtracking solver. This is very rare for generated puzzles but can happen on imported puzzles that require advanced techniques not yet implemented.

4.3 Next step

Same as Hint, but *applies* the step automatically. Chain several "Next step" clicks to watch the puzzle solve itself technique by technique.

4.4 Solve

Fills in the complete solution instantly. The fast solver runs in a background thread and is capped at 2 000 000 search nodes, which guarantees no freeze even on pathologically hard input.

4.5 Check

Compares every user-entered digit against the stored solution. Wrong cells are highlighted in red, correct cells in green. Useful when you're not sure whether a deduction was valid.

4.6 Candidates

Toggles the pencil-mark display. When on, every empty cell shows its current candidate set in small digits. When off, the cells stay clean — useful for a less cluttered view when you're confident about a region.

4.7 Reset

Clears all user entries and restores the puzzle to its generated state. The original clues are preserved.

4.8 File menu

- **New** — generate a new puzzle at the current difficulty.
- **Open** — load an 81-character puzzle string from a `.txt` or `.sdk` file. File size is capped at 10 KB.
- **Save** — save the current grid as an 81-character string (dots for empty cells).
- **Quit** — close the application.

4.9 Language menu

Switches the entire interface between French and English at once. At first launch the language is the system's; the choice made here is remembered.

4.10 Help menu

- **User manual (PDF)** — this document, in the interface language (F1).
- **How to play** — a short in-app guide.
- **Solving techniques** — a full bilingual catalogue of the 8 implemented techniques (see also `guide_techniques.pdf`).
- **Check for updates** — see section 2.
- **Report a problem...** — write a report, sent with the state of the grid.
- **Send problem reports automatically** — the consent asked at first launch.
- **About** — version and credits.

5 Under the hood

5.1 Solver architecture

The solver represents candidate sets as 9-bit masks (`1<<(d-1)` for digit `d`). Every cell knows the bitwise intersection of the allowed digits for its row, column and box. Placing a digit updates the peers' candidate masks in constant time.

Search uses a Minimum Remaining Values (MRV) heuristic: pick the empty cell with the fewest candidates, try each candidate in turn, propagate, recurse. This is the same pattern used by Peter Norvig's well-known solver, extended with an explicit node cap that bounds worst-case effort.

5.2 Human-style techniques

The human solver implements eight techniques, tried in order of increasing difficulty:

Technique	What it finds
Naked Single	Cell with only one candidate left
Hidden Single	Digit that fits in only one cell of a unit
Naked Pair	Two cells sharing exactly two candidates
Naked Triple	Three cells sharing three candidates
Hidden Pair	Two digits that fit in only the same two cells
Pointing Pair	Box candidates aligned on a single row or column
Box/Line Reduction	Line candidates confined to one box
X-Wing	2×2 rectangular elimination pattern

See `guide_techniques.pdf` for worked examples of every technique.

5.3 Generator

Generation starts from a random valid solution (shuffled backtracking), then removes clues symmetrically while verifying after each removal that the puzzle still has exactly one solution. The result is rated by counting clues and running the human solver to identify the hardest technique required. If the rating doesn't match the target difficulty, generation retries up to 40 times before falling back to the closest match.

5.4 Threading and anti-freeze

All long-running work (generation, solving) runs in dedicated `QThread` workers. The UI thread stays responsive and the window never freezes, even if you click "New puzzle" several times in a row or import a pathologically hard grid.

Worker lifecycle is handled defensively: when a thread finishes, its Python wrapper is cleared before Qt's native object is deleted, so subsequent operations never touch a dangling reference.

5.5 Crash reports

Uncaught exceptions are caught by a `sys.excepthook` handler that writes an anonymised JSON report (traceback, versions, system — never the user name nor the home folder) into the data folder, and, with your consent, sends it to the collection point together with native crashes and interface freezes (a watchdog notices when the window stops responding). The data folder is outside the installation folder, which updates replace: `%LOCALAPPDATA%\SudokuSolver` on Windows, `~/Library/Application Support/SudokuSolver` on macOS, `~/.local/share/SudokuSolver` on Linux.

6 Troubleshooting

6.1 "Python not found" on Windows

The launcher couldn't find a working Python interpreter. Install Python 3.10 or newer from https://www.python.org/downloads/ and **make sure to tick "Add Python to PATH"** during installation. Then re-run `launch.bat`.

6.2 "Installation failed" after a Python upgrade

If you upgraded Python and the old venv is stale, run:

```
launch.bat --repair
```

This deletes the virtual environment and reinstalls everything cleanly.

6.3 The generator is slow on Expert

Expert puzzles that specifically require X-Wing are rare, so the generator may retry up to 40 times before falling back to a Hard-level puzzle with a slightly more constrained clue count. This is by design, and the background thread keeps the UI responsive throughout.

6.4 I imported a puzzle and Solve says "no solution"

Either the imported grid has a typo (wrong digit in a given), it already contains a conflict (check the red highlights), or it genuinely has no solution. The solver also bails out after 2 000 000 search nodes — this almost never triggers on valid 9×9 grids.

6.5 Where are my settings stored?

The only setting, the language, is kept by Qt in the user's preferences (registry on Windows, `~/.config` on Linux, preferences on macOS). The log and the reports live in the data folder described in the *Crash reports* section; nothing is written into the installation folder.

6.6 The app crashed — what should I do?

If you accepted automatic reports, there is nothing to do: the report has been sent. Otherwise, use **Help > Report a problem...** and describe what you were doing; without a connection, the report is saved as a `.zip` on the desktop, ready to be attached to a GitHub issue.

7 FAQ

Can I play on a phone?

No — this is a desktop application. For mobile, use any Sudoku app from your platform's store.

Does it phone home?

Only for two things: the update check (GitHub releases API, once at start-up) and, if you agreed, problem reports. No analytics, no grid data unless you send a written report.

What Sudoku formats are supported?

81-character strings with digits `1-9` and any of `.`, `0`, `-` or space for empty cells. Line breaks are tolerated.

Why is the "Solving techniques" guide a separate PDF?

Because it is a reference you want to browse while solving. The main manual (this document) is the "how to use the app" guide; the techniques guide is the "how to solve Sudoku" reference.

8 Credits

SudokuSolver is MIT-licensed. Thanks to Peter Norvig for the classic constraint-propagation solver design, and to the Sudoku solving community for decades of accumulated technique catalogues.

Issues and contributions welcome at:
https://github.com/ARP273-ROSE/SudokuSolver